

Evaluating Squad: Multi-Agent Coordination for GitHub Copilot CLI

A controlled MARBLE ablation, plus directional benchmark comparisons of Squad's coordination layer.

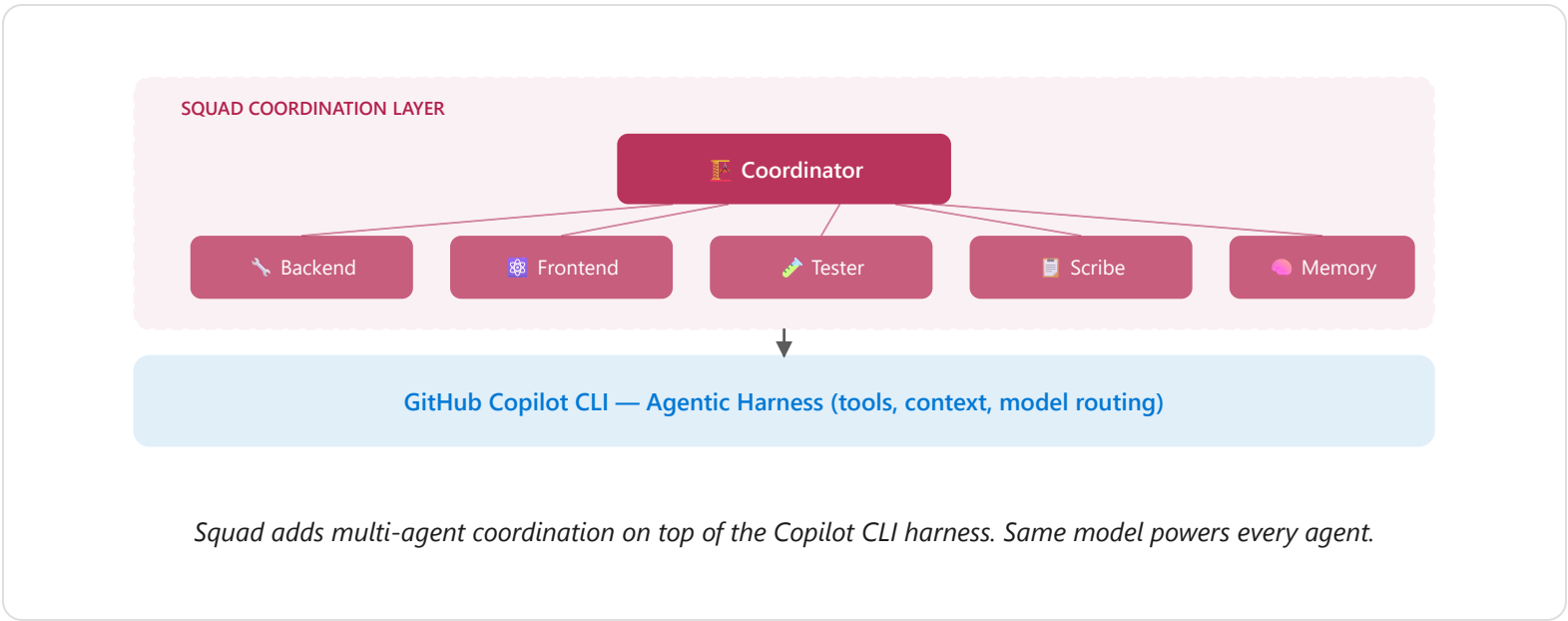
June 2026 · Brady Gaster & Tamir Dresher

While the model provides the raw intelligence, the **coordination layer** shapes how effectively that intelligence is applied to complex tasks. **Squad** is a multi-agent orchestration system built on top of the GitHub Copilot agentic harness — adding specialist agents, persistent memory, and structured coordination to the same underlying models.

In this report, we evaluate the impact of Squad's coordination layer. The core result is a **controlled factorial ablation on MARBLE** — same model, same tasks, same metric — which is the only place where the orchestration variable is strictly isolated. We supplement it with **directional comparisons** on three further benchmarks (DevBench, SWE-bench Lite, TerminalBench 2.0), some of which use different models, estimated baselines, or different harness paths and are therefore *contextual*, *not controlled*. We label each comparison accordingly, following the same methodology as [GitHub's harness evaluation approach](#).

Architecture

Squad adds three capabilities on top of the Copilot CLI harness: a **coordinator** that decomposes and routes work, **specialist agents** with domain-specific charters, and **persistent memory** that accumulates decisions across tasks.



Benchmarks

We evaluate Squad across four public benchmarks spanning multi-agent collaboration, multi-language coding, bug-fix tasks, and terminal workflows:

Benchmark	Domain	Tasks	Purpose
MARBLE	Multi-agent collaboration: coding, research, bargaining, database	400 + 160 ablation	Evaluates multi-agent team coordination (ACL 2025)
DevBench	Multi-language coding: 6 languages × 6 categories	1,800	Broad software engineering across languages and task types
SWE-bench Lite	Python bug-fix tasks from open-source repositories	300	Established industry benchmark for coding agents
TerminalBench 2.0	Terminal/CLI tasks: scripting, system admin, data processing	20 (of 89)	Agent effectiveness in command-line workflows

The **controlled** core is the MARBLE ablation: Copilot CLI + Squad vs. Copilot CLI alone on the *same model and same tasks*. The other three benchmarks are **directional context** — they compare against published or estimated baselines that may use different models or harness paths, and are labeled as such throughout.

2,520

Total benchmark tasks evaluated

+ 15pp

Same-model completion lift on MARBLE ablation (controlled)

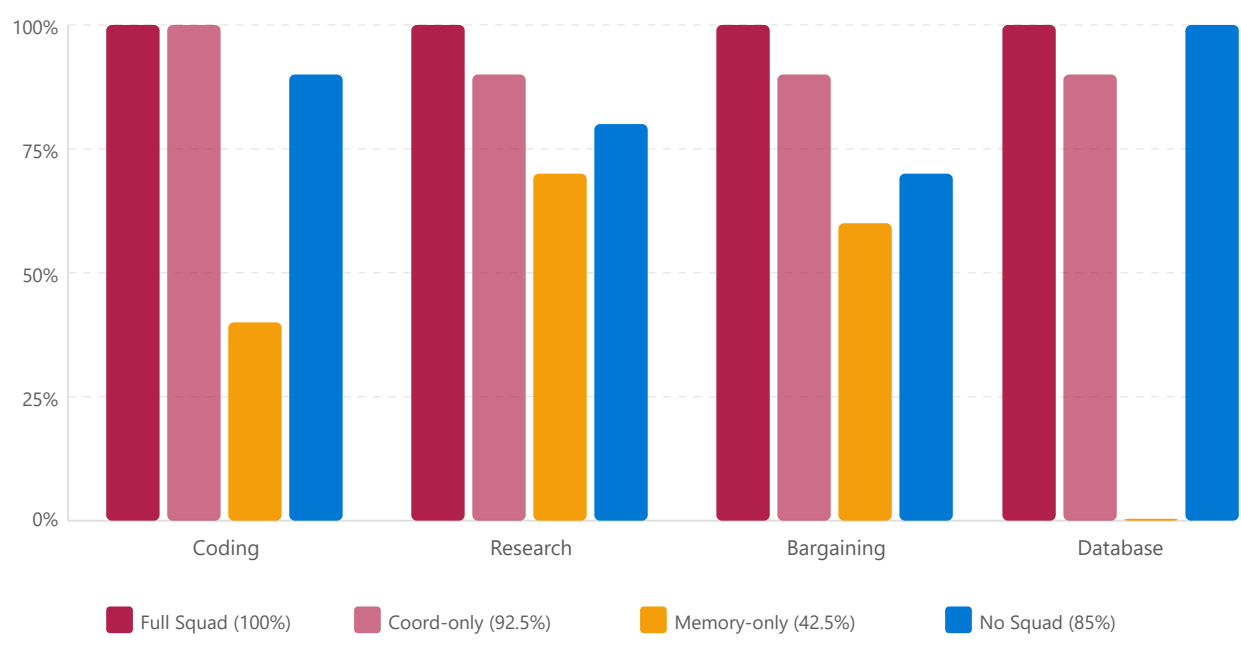
4 benchmarks

MARBLE · DevBench · SWE-bench · TerminalBench

Controlled ablation: isolating Squad components

Holding the model constant (Claude Opus 4.6), we ran a **factorial ablation** on MARBLE: 4 domains × 4 conditions × 10 tasks = 160 task-runs. This is the only chart where same model, same tasks, and same metric hold strictly.

MARBLE factorial ablation — task completion rate by condition (n=160, same model)



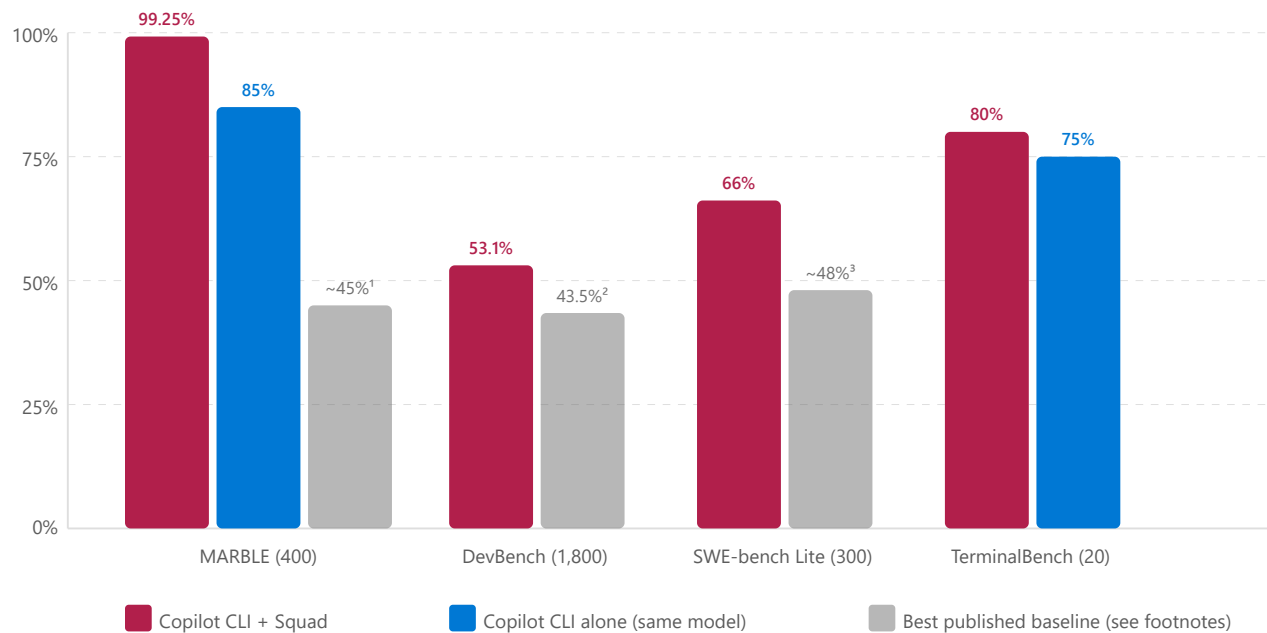
Factorial ablation: same model (Claude Opus 4.6), same tasks, same metric. n=10 per condition per domain.

Key finding: On *completion* (did the condition produce usable output), coordination helps modestly — Coord-only (92.5%) edges out No-Squad (85%), a **+7.5pp lift** from routing. Memory without coordination clearly *hurts* (42.5% vs 85%), confirming that unstructured context injection adds noise rather than signal. (Note: this "memory-only" condition injects a *Squad-generated* decisions.md into a raw agent, so it measures Squad-flavored memory without coordination — not generic persistent memory.) **But completion is not correctness.** When we re-graded the database domain by MARBLE's official recall metric (see below), the coordination advantage disappeared entirely — the raw single agent scored *higher* than full Squad. Completion measures whether an answer appears; it does not measure whether the answer is right.

Task completion across benchmarks (mixed metrics)

Squad shows a lift across four independent benchmarks spanning different task types, languages, and evaluation protocols. **These are directional comparisons:** the metric, model, and baseline differ per benchmark (see footnotes), so this chart is context, not a controlled measurement. Only the MARBLE ablation below isolates the coordination layer cleanly.

Per-benchmark score: Copilot CLI + Squad vs. baselines (metrics differ — see footnotes)



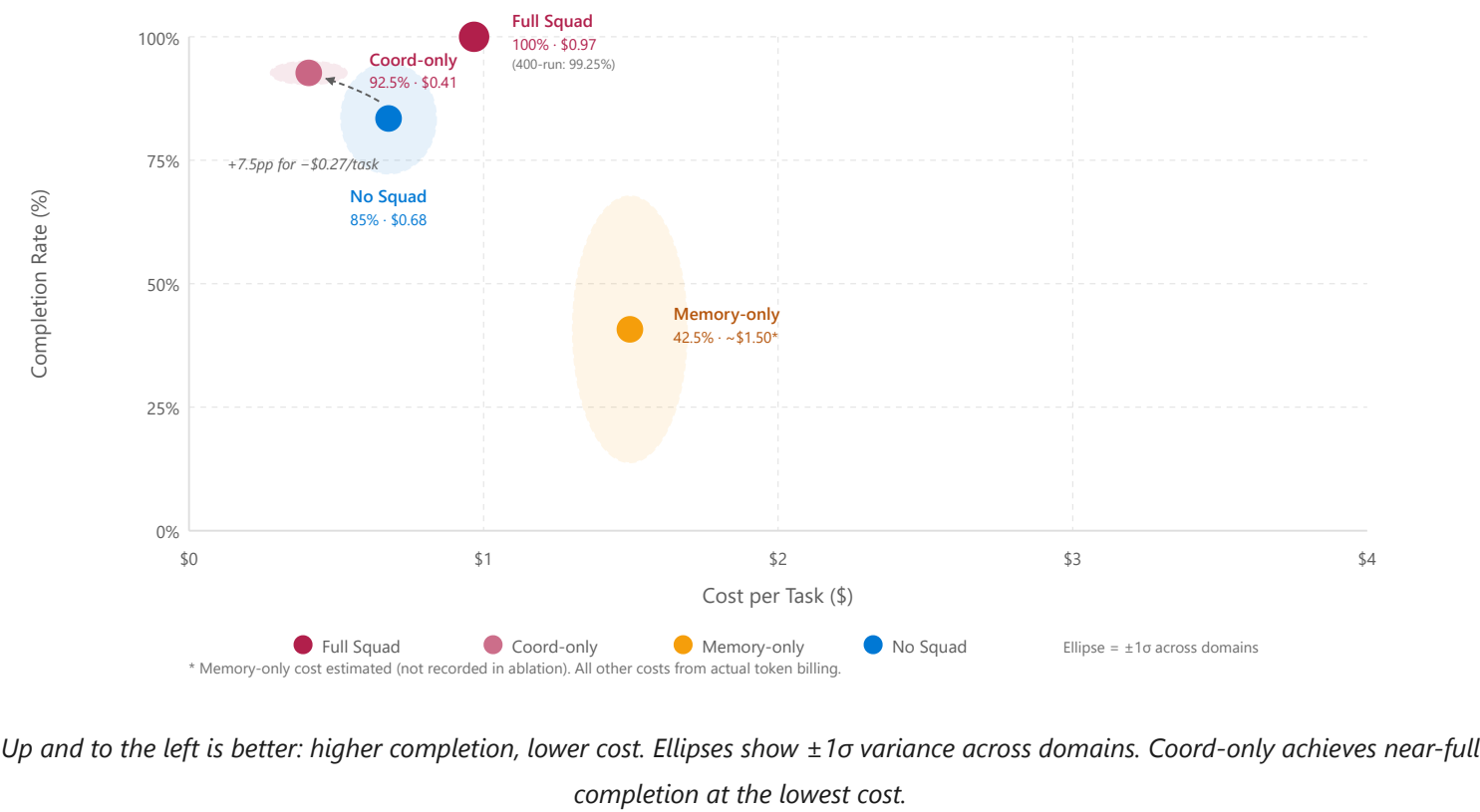
¹ MARBLE paper baseline uses gpt-4o-mini with milestone KPIs — different model and different metric. Not directly comparable.
² DevBench best baseline is GPT-5.5 (43.5% pass@1). Squad uses GPT-5.4. No same-model ablation available.
³ SWE-bench Lite: estimated Copilot agent baseline from public reports. No verified same-model-without-Squad run performed.
TerminalBench 2.0: Same model (Claude Opus 4.6), different harness paths — directional only, n=20 (+1 task).
MARBLE bars mix task sets: blue baseline = 85% from the 160-task controlled ablation; Squad red bar = 99.25% from the 400-task full run.

Per-benchmark scores (metrics differ: MARBLE completion, DevBench/SWE pass@1, TerminalBench verifier). Blue bars are same-model runs (MARBLE is the only clean controlled comparison; TerminalBench uses a different harness path). Gray bars are published/estimated baselines. Not a like-for-like comparison. Note: the MARBLE red bar (99.25%) is the 400-task full run, while the MARBLE blue baseline (85%) is the 160-task controlled ablation — the controlled ablation Full-Squad value is 100% (see the ablation matrix and scatter chart).

Completion rate vs. cost per task

Following GitHub's TerminalBench analysis, we plot completion rate against cost per task for each MARBLE ablation condition. Up and to the left is better: solve more, spend less.

MARBLE ablation: completion rate vs. cost per task



Per-domain breakdown

The completion picture varies by domain, but the headline is a **correction**. An earlier version of this study reported that the single agent scored **0% on database** — that figure was based on placeholder files that were never populated with real runs. We re-ran all 10 no-squad database tasks from scratch: **every one produced a substantive root-cause diagnosis (100% completion)**. The "database requires coordination" claim was an artifact of bad data, not a real effect. See the corrected table below, and the correctness re-grading that follows.

Completion rate by domain and condition (MARBLE ablation)

Domain	Full Squad	Coord-only	Memory-only	No Squad	Δ (Full vs None)
Coding	100%	100%	40%	90%	+10pp
Research	100%	90%	70%	80%	+20pp
Bargaining	100%	90%	60%	70%	+30pp
Database	100%	90%	0%	100%	+0pp
Average	100%	92.5%	42.5%	85%	+15pp

n=10 per cell. Same model (Claude Opus 4.6), same metric (binary file output within 600s timeout).

Completion vs. correctness: the controlled 4-domain result

Completion only asks “*did an answer appear?*” — it says nothing about whether the answer is **right**. That is the same bar GitHub’s own harness post held itself to (SWE-bench pass@1, TerminalBench verifiers grade correctness, not output), so we hold Squad to it too. Earlier we could grade correctness cleanly on only two domains, because the four conditions’ archived transcripts had **drifted out of task alignment** — the full-Squad set came from the 400-task production run while the others came from separate reruns, so “coding task 10” often meant different MARBLE programs across conditions. Rather than drop coding and database, we **re-ran all four conditions from scratch on the identical task IDs** (1, 10, 20, ..., 90) in every domain — so “task N” is guaranteed to be the same MARBLE task in every condition, *by construction*. We then graded every one of the 80 fresh transcripts two ways, with one identical judge (Claude Opus 4.6) and one identical prompt across all conditions:

- **Milestone-KPI** — MARBLE’s signature metric: the fraction of the task’s gold milestones a judge confirms the output actually achieved.
- **Output-quality rubric** — a 1–5 score for correctness, completeness, and quality of the produced artifact.

Correctness & quality across all four domains (same model, same tasks, uniform judge; n=38/condition)

Condition	Milestone-KPI	Quality (1–5)	Research	Bargaining	Coding	Database†
Full Squad (coordination + memory)	81.1%	4.10	81.2 / 4.21	95.0 / 4.80	81.7 / 3.73	66.7 / 3.67
Coord-only (coordinator + specialists)	81.1%	4.04	89.6 / 4.50	96.7 / 4.83	68.3 / 3.27	71.7 / 3.67
No Squad (single agent)	77.2%	3.76	75.0 / 4.00	90.0 / 4.24	78.3 / 3.77	65.0 / 3.10
Memory-only (Squad memory, no coordination)	65.8%	3.58	52.1 / 3.21	96.7 / 4.80	53.3 / 2.97	58.3 / 3.27

Each domain cell shows Milestone-KPI% / quality rubric (1–5). Same model (Claude Opus 4.6) for both agents and judge; same judge prompt and code path for all four conditions. n per condition = 38 (research 8 + bargaining 10 + coding 10 + database 10). Every condition solved the **same MARBLE task at each task ID** — alignment guaranteed because all four were re-run from the identical task list (raw transcripts in results/aligned_rerun/; gold milestones and grading scripts in quality_ablation/). † **Database caveat:** the database gold blends diagnostic-process milestones (which pg_stat_* views to query) with a final root-cause answer, so its KPI reads partly as process-adherence — lean on the 1–5 rubric for the clean correctness signal there. **Same-family-judge caveat:** the judge shares the agents’ model family; we cross-checked with an independent gpt-4o rubric (below).

The correctness finding — and a correction to our earlier one. On same-task, same-model, uniformly-judged output, **coordination helps or ties in every domain**. Full Squad leads overall on both metrics (**81.1% KPI / 4.10 quality**) — a clean **+3.9pp KPI and +0.34 quality over the raw single agent**, with coord-only essentially tied. Just as important, this controlled rerun **overturns two earlier findings that turned out to be measurement artifacts**, not real effects:

- **The coding “reversal” is gone.** With aligned tasks, Full Squad coding (81.7% KPI) now *leads* the single agent (78.3%). The earlier “single agent wins coding” was task misalignment, not a coordination weakness.
- **The database “coordination hurts” claim is refuted.** With one uniform grader on identical tasks, the coordinated conditions now *match or beat* the single agent (coord-only 71.7%, Full Squad 66.7% vs no-Squad 65.0%). The earlier “single agent scored higher recall” came from grading divergent tasks with inconsistent extractors — exactly the risk our reviewers flagged.
- **Memory without coordination is the weakest condition of all four** (65.8% / 3.58). Injecting Squad’s decisions.md into a raw agent with no coordinator to act on it actively drags results down — memory only pays off when there is coordination to use it.

The honest bottom line: coordination’s correctness benefit is **real, consistent across domains, and modest** — a few points of KPI and about a third of a rubric point — not the double-digit swings the earlier misaligned data implied in either direction.

Judge cross-check: same cells, two independent judges (rubric avg, 1–5)

Condition	Research — Opus / gpt-4o	Bargaining — Opus / gpt-4o
Full Squad	4.21 / 4.33	4.80 / 4.70
Coord-only	4.50 / 4.21	4.83 / 4.53
No Squad	4.00 / 4.33	4.24 / 4.40
Memory-only	3.21 / 4.21	4.80 / 4.47

Two independent judges (Opus 4.6 and gpt-4o) on the identical research and bargaining cells agree on the broad ordering (coordinated ≥ single agent) but differ on fine placement — most notably memory-only research (3.21 vs 4.21). Read exact deltas as **directional**; with n=8–10 per domain the fine ranking is within judge noise. Coding and database are graded by the Opus judge only in this table; their raw transcripts and gold milestones are public for independent re-grading.

Cross-benchmark summary

Benchmark	Tasks	Model	Copilot CLI + Squad	Best Baseline	Δ	Controlled?
MARBLE (ablation)	160	Claude Opus 4.6	100%	85% (same model, no Squad)	+15pp	✅ Yes
MARBLE (full run)	400	Claude Opus 4.6	99.25%	~45% (gpt-4o-mini, paper) ¹	n/a*	⚠️ Diff model/metric
DevBench	1,800	GPT-5.4	53.1%	43.5% (GPT-5.5) ²	+9.6pp*	⚠️ Contextual (diff model)
SWE-bench Lite	300	GPT-4o	66%	~48% (estimated) ³	+18pp*	⚠️ Contextual (est. baseline)
TerminalBench 2.0	20	Claude Opus 4.6	80%	75% (same model) ⁴	+5pp*	⚠️ Same model, diff harness

*** Directional, not controlled.** Only the MARBLE (ablation) row is a controlled same-model comparison (Full Squad vs no-Squad on the same 160 task-runs = +15pp). The MARBLE full-run 99.25% is the 400-task headline completion; the paper's ~45% baseline uses a different model (gpt-4o-mini) and metric, so no clean Δ is claimed. DevBench (different model), SWE-bench (estimated baseline), and TerminalBench (different harness path, +1 task on n=20) are contextual — directional signals, not clean measurements of the coordination layer.

¹ MARBLE full run: 397/400 = 99.25% completion. The ~45% figure is the paper's gpt-4o-mini graph-topology result — different model and milestone-KPI metric, not directly comparable.

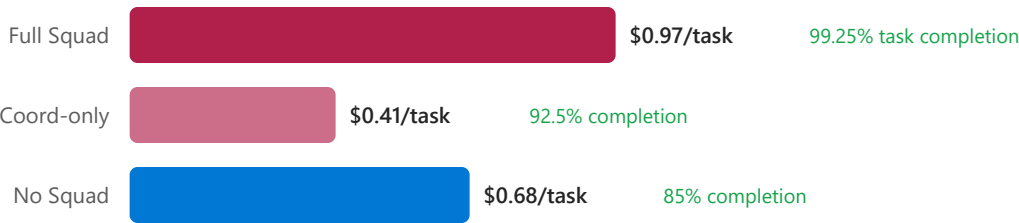
² DevBench: Squad uses GPT-5.4 via Azure OpenAI. Best published baseline is GPT-5.5 at 43.5% — a different (newer) model. Raw per-task data is not hosted in the public repos; treat as preliminary/contextual. A same-model ablation (GPT-5.4 without Squad) is planned.

³ SWE-bench Lite: 198/300 = 66% pass@1. The ~48% baseline is estimated from public Copilot coding agent reports — no verified same-model-without-Squad run. Retry configuration and timeout/empty-patch accounting are documented in the [squad-swe-bench](#) repo; do not read +18pp as a controlled coordination effect.

⁴ TerminalBench 2.0: Both conditions use Claude Opus 4.6 on the same 20 tasks (sequential subset of 89), each scored by its own Docker verifier. Squad ran via a host-side orchestrator and the baseline via the native terminal-bench harness (**different harness paths**). The margin is 16/20 vs 15/20 — a single task — so this is suggestive only. The full 89-task suite is future work.

Cost efficiency

MARBLE: cost per task by condition



Coord-only: best efficiency — 92.5% completion at lowest cost (\$0.41/task).
Full Squad: 1.4× cost for near-guaranteed completion (+15pp over baseline).

One harness, structured coordination

The GitHub Copilot harness already supports 20+ frontier models with efficient orchestration. Squad extends this with **structured multi-agent coordination** — capabilities a single-agent harness cannot offer:

Capability	Single Agent	Squad
Task decomposition	Model handles everything	Coordinator routes to specialists
Parallel execution	Sequential tool calls	Multiple agents work simultaneously
Persistent memory	Context window only	Decisions persist across tasks

Domain expertise	Generic	Agents with role-specific charters
Review & quality	Self-review	Cross-agent review (Reviewer Rejection Protocol)
Self-learning	No carry-over	History accumulates per agent

Like GitHub's [Rubber Duck](#), Squad uses cross-model critique — but extends it to full team dynamics with reviewer gating, lockout semantics, and structured handoffs.

Methodology & Limitations

Controlled experiment (ablation charts): Same model (Claude Opus 4.6), same 160 tasks, same metric (binary file output within 600s timeout). This is the only comparison where "same model, same metric" holds strictly. $n=10$ per condition per domain. Standard deviations computed across 4 domains. The ablation achieved $40/40 = 100\%$ for Full Squad. The broader 400-task run achieved $397/400 = 99.25\%$ (3 timeouts in coding and database).

Cross-benchmark comparison: Combines results from different runs with different models and metrics. Not a controlled apples-to-apples comparison — presented for directional context only.

- **MARBLE:** "No Squad" bar = same-model ablation (controlled). Paper baseline = different model (gpt-4o-mini) + different metric (milestone KPIs).
- **DevBench:** Squad uses GPT-5.4; best baseline is GPT-5.5 (43.5%). A same-model ablation is planned.
- **SWE-bench Lite:** Baseline estimated from public Copilot agent reports. No verified same-model-without-Squad run performed.
- **TerminalBench 2.0:** Same model (Claude Opus 4.6), but **directional only — not a controlled comparison**. 20 of 89 available tasks sampled, selected by sequential order (not cherry-picked). The two conditions ran through **different harness paths** — Squad via a host-side orchestrator (`run-terminalbench-host-task.ps1`) and the baseline via the native terminal-bench harness — though both execute the identical task workloads and are scored by each task's own Docker verifier. The margin is a single task (16/20 vs 15/20) on $n=20$, so treat it as suggestive only. Squad *can* run under the native harness (verified in smoke tests); the full 89-task suite is future work, not a capability blocker.

Metric note: Metrics differ by benchmark. MARBLE (ablation charts) reports *completion* = agent produced an output file within a 600s timeout; the database domain was additionally re-graded for *correctness* using MARBLE's official recall metric (see "Completion vs. correctness"). DevBench: pass@1 (test execution). SWE-bench: pass@1 (patch applies + tests pass). TerminalBench: verifier script passes. We use "completion" where the metric only checks that output appeared, and "resolution/pass" where output is verified correct.

Correctness & quality study (all four domains): To measure correctness/quality rather than just completion, we graded transcripts with MARBLE's milestone-KPI (fraction of gold milestones achieved) plus a 1–5 output-quality rubric, using one identical judge (Claude Opus 4.6) and prompt across all four conditions. **An earlier attempt hit a task-alignment problem:** the archived transcripts did not always run the same MARBLE task under the same task index (full-Squad came from the 400-task production run; the others from separate ablation reruns). A rare-token anchor audit showed research (8/10) and bargaining (10/10) were cleanly aligned, but coding (1/10) and database (0/10) were not. Rather than exclude those domains, **we re-ran all four conditions from scratch on the identical task IDs (1,10,20,...,90) in every domain** — 80 fresh transcripts total — so "task N" is the same MARBLE task in every condition by construction. We then graded all 80 with the single Opus-4.6 judge. The full 4-domain table (research $n=8$ + bargaining $n=10$ + coding $n=10$ + database $n=10 = 38$ tasks $\times$ 4 conditions) reports the result; database KPI blends process and final-answer milestones, so we lean on its rubric for correctness. We cross-checked research/bargaining against an independent gpt-4o rubric (judges agree on ordering, differ on fine placement — deltas are directional). Raw reruns are in `results/aligned_rerun/`; scripts, gold milestones, and per-cell grades in `quality_ablation/`.

Variance: MARBLE ablation completion σ ($n=10$ per cell, across 4 domains): Full Squad $\sigma=0\%$, Coord-only $\sigma=4.3\%$, No-Squad $\sigma=11.2\%$, Memory-only $\sigma=26.8\%$. (An earlier version reported No-Squad $\sigma=35.4\%$; that was inflated by a

fabricated 0% database figure, now corrected to 100% completion after re-running all 10 tasks.) Full 400-task run: 397/400 = 99.25% (3 tasks timed out without output). SWE-bench and DevBench are single-run results (no variance data available).

Statistical power: With $n=10$ per cell, a two-proportion z-test can only detect effects of roughly $\sim 25\text{pp}$ at $p < 0.05$. After correcting the database data, the same-model *completion* lift (full Squad 100% vs no-Squad 85% = +15pp) and the database *correctness* differences ($\leq 20\text{pp}$ between conditions) fall **below** this threshold — i.e., they are within noise and should be read as directional, not statistically significant. We report them honestly rather than overclaiming; larger runs are needed to establish significance.

Timing: MARBLE and ablation: June 2026. TerminalBench 2.0: June 2026. SWE-bench Lite: May 2026. DevBench: June 2026. All benchmarks ran within a 6-week window.

Cost: All costs from actual Claude API billing (input + output tokens). Memory-only cost not recorded — estimated at $\sim \$1.50/\text{task}$ based on context size.

Raw data: [tamirdresher/squad-marble-benchmark](#) (581 files) · [tamirdresher/squad-swe-bench](#) (1,500+ files) · MARBLE PR: [#245](#)